zotregistry.dev

None

None

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

Table of contents

1. What's New 5
1.1 v2.1.23 5
1.2 v2.1.22 5
1.3 v2.1.21 7
1.4 v2.1.20 8
1.5 v2.1.19 8
1.6 v2.1.18 8
1.7 v2.1.17 9
1.8 v2.1.16 10
1.9 v2.1.15 11
1.10 v2.1.14 12
1.11 v2.1.13 12
1.12 v2.1.12 13
1.13 v2.1.11 13
1.14 v2.1.10 13
1.15 v2.1.9 14
1.16 v2.1.8 14
1.17 v2.1.7 14
1.18 v2.1.6 14
1.19 v2.1.5 15
1.20 v2.1.4 15
1.21 v2.1.3 15
1.22 v2.1.2 16
1.23 v2.1.0 16
1.24 v2.0.4 16
1.25 v2.0.3 16
1.26 v2.0.2 16
1.27 v2.0.1 17
1.28 v2.0.0 17
1.29 v1.4.3 19
2. General 20
2.1 Concepts 20
2.2 Summary of Key Features 23
2.3 Architecture 0
2.4 Extensions 0

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

2.5 Released Images for zot 0
2.6 Glossary 0
2.7 About the zot Project 0
3. User Guides 0
3.1 Push and Pull Image Content 0
3.2 Use the Web Interface to Find Images 0
3.3 Using the command line interface (zli) 0
4. Installation Guides 0
4.1 Installing zot on Bare Metal Linux 0
4.2 Installing zot with Kubernetes and Helm 0
5. Administrator Guides 0
5.1 Getting Started with zot Administration 0
5.2 Configuring zot 0
6. Developer Guide 0
6.1 Onboarding zot for Development 0
6.2 Developing New Extensions 0
6.3 Using the zot API 0
6.4 zot API Command Reference 0
6.5 Contributing to zot Development 0
7. Articles 0
7.1 Building an OCI-native Container Image CI/CD Pipeline 0
7.2 User Authentication and Authorization with zot 0
7.3 Verifying image signatures 0
7.4 Immutable Image Tags 0
7.5 Software Provenance Workflow Using OCI Artifacts 0
7.6 zot Security Posture 0
7.7 Storage Planning with zot 0
7.8 Configuring zot Tag Retention Policies 0
7.9 OCI Registry Mirroring With zot 0
7.10 zot Clustering 0
7.11 Scaling zot 0
7.12 Deploying a Highly Available zot Registry 0
7.13 Monitoring the registry 0
7.14 Using GraphQL for Enhanced Searches 0
7.15 Benchmarking zot with zb 0
7.16 Performance Profiling in zot 0
7.17 Using `kind` for Deployment Testing 0
7.18 *containerd* Mirroring From *zot* 0

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

7.19 Docker Users Guide to zot 0

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1. What's New

1.1

1.1.1 Highlights

- **Streaming sync:** opt-in `stream` serves blobs while downloading from a verified HTTPS upstream. See Mirroring.
- **Repository quotas:** `storage.maxRepoBytes` limits logical bytes per repository; pushes exceeding the limit return `413`. See Storage.
- **Access control:** `${username}` repository patterns provide per-user namespaces, and `disableAnonymousCatalog` can require authentication for catalog listing without disabling anonymous pulls. See Authorization.
- **OpenID/OIDC:** provider outages no longer block startup, cached workload signing keys survive refresh failures, and OpenID users receive locally configured groups. Browser login supports a custom `discoveryurl`.
- **zli TLS:** profiles support private CAs and mTLS through `ca-cert`, `client-cert`, and `client-key`. See zli.
- **Runtime/API:** graceful shutdown defaults to a 30-second drain timeout, startup failures return a nonzero exit status, and blob uploads accept `digest-algorithm`.

1.1.2 Reliability and Upgrade Notes

- Storage fixes protect concurrent uploads and manifest updates, improve dedupe repair and crash durability, and prevent premature garbage collection.
- Retention preserves untagged activity statistics; storage walks are faster. CVE searches include multi-architecture images without reporting unscannable attestations as errors.
- Sync handles upstream catalog pagination correctly and loads all CA certificates in `certDir`.
- **Upgrade impact:** empty or catalog-only bearer grants no longer expose repository data through search; clients need explicit repository grants.

1.1.3 eMudhra CA Distrust

The incident is addressed by updated Debian/Distroless CA bundles that remove **emSign ECC Root CA - G3** and **AAA Certificate Services**, blocking both compromised TLS chains. Ensure your deployment uses the updated bundle and keeps TLS verification enabled. See Outbound TLS trust.

See the full changelog for all changes.

1.2

1.2.1 Storage Outage and Data-Integrity Handling

- Storage drivers now distinguish missing content from transient failures and permanent errors across local, S3, GCS, and Azure storage.
- Registry API requests return `503 Service Unavailable` for transient storage failures and `500 Internal Server Error` for permanent failures instead of reporting misleading `404`, `400`, or `403` responses.
- Garbage collection, deduplication, scrub, and storage enumeration fail closed when a backend cannot provide a complete view. The affected operation aborts and can retry on its next cycle instead of pruning content based on an assumed-empty result.
- Re-uploading a manifest repairs equal-size corruption, including remote dedupe origins. Additional fixes prevent remote dedupe self-link and concurrent-upload races from emptying shared content or returning spurious `500` responses.
- Garbage collection now completes for repositories that do not have a MetaDB record, and an overwritten tag's previous digest remains pullable as an untagged manifest until retention and garbage collection remove it.
- Corrupt S3 dedupe stubs no longer return a successful empty blob response.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

See Storage.

1.2.2 Sync and Multi-Architecture Images

- Periodic sync can use `platforms` at the registry or content-rule level to copy only selected `os/arch[/variant]` children of a multi-architecture index.
- Sync now preserves upstream digests. The `preserveDigest` setting is deprecated and ignored; syncing Docker schema 2 content requires `http.compat: ["docker2s2"]`.
- `onDemandInBackground` can return a local `404` immediately while filling the image asynchronously, allowing clients configured with another registry to fail over without waiting.
- Hard on-demand upstream failures now return `503` rather than falling through to `404`, while genuine upstream misses remain `404`.
- On-demand sync reuses locally stored layers before fetching remotely, and search metadata updates correctly as sparse multi-architecture images are populated.
- AWS ECR sync credential helpers can again run `credential_process` commands correctly.

See OCI registry mirroring and OCI registry mirroring.

1.2.3 Authentication and Security

- **Upgrade impact:** when authentication is enabled, profiling and trust-key upload endpoints now require an explicit matching `accessControl.adminPolicy`. Without an admin policy, authenticated users can still receive repository access but cannot use these admin-only routes.
- API keys, OpenID sessions, workload OIDC tokens, and traditional bearer authentication can coexist. zot's token endpoint exchanges locally issued API keys and can forward only unknown credentials to a configured upstream token service.
- zot-issued API keys are never forwarded to an upstream token endpoint.
- HTTPS responses include `Strict-Transport-Security: max-age=63072000; includeSubDomains`.
- Anonymous-only UI deployments no longer return `500` when checking for a session.
- Legacy cosign signature and SBOM tag recognition now requires the complete canonical tag form.

See Authentication and authorization and Configuring zot.

1.2.4 Runtime, API, and Compatibility Improvements

- `zot healthcheck` (alias `zot ready`) probes `/livez`, `/readyz`, or `/startupz` without requiring a shell or `curl`, including in distroless containers.
- zot notifies systemd when startup has completed and when shutdown begins; service units can use `Type=notify`.
- Manifest pushes now reject invalid Distribution Specification tags with `400 MANIFEST_INVALID`. Existing non-conformant tags remain readable and deletable.
- A manifest that references a missing blob returns `MANIFEST_BLOB_UNKNOWN` with the missing digest.
- Catalog and tag pagination continue correctly if the `last` cursor was deleted, and large tag-list page sizes no longer overflow.
- Missing-repository manifest deletes now return `404`, missing nested-repository referrers return an empty index, and streamed upload `Range` responses cover the complete upload.
- Cosign attestations are classified as referrers rather than signatures and no longer make an image appear signed.

See Installing zot on bare metal Linux and Push and pull image content.

✏️ Signature and attestation layer payloads are no longer stored in MetaDB. This prevents large metadata records, but existing BoltDB files do not shrink automatically. Compact reclaims space only after rewritten records leave freelist pages; a full `meta.db` rebuild is a separate, more disruptive option. See Verifying image signatures. Avoid downgrading to a release that expects signature content to be present in MetaDB.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.3

1.3.1 Sync Throughput and Upstream Check Controls

- On-demand sync can set `manifestCheckInterval` to serve a locally cached tag without checking the upstream registry again until the interval expires.
- `reqConcurrent` and `reqPerSec` configure per-host upstream concurrency and request-rate limits.
- `disableHTTP2` and `maxIdleConnsPerHost` can improve throughput on high-bandwidth links by using a pool of parallel HTTP/1.1 connections.
- Concurrent on-demand requests for the same image are now coalesced into one upstream sync operation.

See OCI registry mirroring.

1.3.2 Storage and Distribution Behavior

- **Upgrade impact:** blob `HEAD` and ranged `GET` requests are repository-local by default. A digest that exists only in another repository's dedupe cache now returns `404` instead of being materialized into the requested repository. Set `storage.hydrateBlobOnRead` to `true` only when the previous behavior is required.
- Cross-repository dedupe mounts now require read access to a source repository that contains the blob, as well as create access to the destination repository.
- Garbage collection now correctly handles multi-tag manifests, shared cosign referrers, nested indexes, and Docker-compatible media types without repeated exponential index walks.
- Dedupe and storage walks now cover nested repository namespaces and avoid redundant full-storage traversals.

See Storage.

1.3.3 Runtime and Deployment Improvements

- zot supports systemd socket activation, including binding privileged ports without granting the zot process `CAP_NET_BIND_SERVICE`.
- Configuration files mounted from Kubernetes ConfigMaps and `htpasswd` files mounted from Secrets are detected after atomic symlink replacement and reloaded without restarting the pod.
- Event sink configuration can now be enabled, disabled, or changed by a configuration reload.

See Installing zot on bare metal Linux, Configuring zot, and Authentication and authorization.

1.3.4 Scanning, Signing, Events, and Management

- Trivy scans can use a configured ignore file through `extensions.search.cve.trivy.ignoreFile`.
- Image trust recognizes and verifies the Sigstore bundle format produced by cosign v3 (`application/vnd.dev.sigstore.bundle.v0.3+json`).
- Event publishers can override the CloudEvents `source` and event-type namespace with `source` and `typePrefix`.
- The management endpoint reports `http.auth.allowAnonymousAccess: true` when any repository has an `anonymousPolicy`, allowing clients such as the zot UI to offer guest access without probing the registry.

See CVE scanning, Verifying image signatures, and Events.

1.3.5 Reliability and Compatibility Fixes

- Docker schema 2 media types are preserved through storage metadata, search, delete, prune, and scrub operations.
- SBOM generation no longer broadens Trivy vulnerability detection or produces additional CVE results.
- Repository quota slots are released when the final manifest is deleted from a repository.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

- Additional fixes improve Kubernetes-mounted authentication reloads, sync cleanup, event delivery logging, metrics lifecycle, download statistics, and storage locking under concurrent uploads.

1.4

This release primarily restores the downloadable binaries that were not published with v2.1.19. Download the binaries from the v2.1.20 GitHub release. This release also includes UI improvements from an updated `zui` version.

1.5

1.5.1 Workload Identity and Sync Authentication

- OIDC workload identity tokens can now complete the registry token-service login flow, including Kubernetes `imagePullSecret` authentication.
- Sync adds OAuth2 JWT assertion and RFC 8693 token-exchange credential helpers for short-lived upstream credentials.
- A new `gcp` credential helper uses Google Application Default Credentials to mirror from Google Artifact Registry and compatible registries.

See Authentication and authorization and OCI registry mirroring.

1.5.2 Retention and Garbage Collection Controls

- Retention policies can protect actively used untagged manifests, such as digest-only pull-through cache entries, with `keepUntagged`.
- Periodic garbage collection can be restricted to a daily UTC `gcTimeWindow`, including windows that cross midnight.

See Tag retention policies and Storage.

1.5.3 Signing, Scanning, Events, and GraphQL

- Generated notation trust policies now support the `tsa:default` trust store for timestamped signatures.
- Trivy scanning now accepts OCI zstd-compressed layers.
- Successful, non-cached image scans emit `zotregistry.image.scanned` events with vulnerability summaries.
- GraphQL `ImageSummary` now exposes `ArtifactType`.

See Verifying image signatures, CVE scanning, Events, and GraphQL.

1.5.4 Reliability, Conformance, and Security Fixes

- Hardened the OIDC UI callback redirect against open redirects that used a backslash in the target URL.
- Sync registries can set `maxRetryDelay` for exponential HTTP retry backoff (when greater than `retryDelay`).
- Improved OCI conformance for malformed manifest references, referrers responses, and blob digest errors.
- Additional fixes cover digest multi-tag overwrite authorization, cosign verification binding, garbage collection of incomplete content, storage integrity checks, sync diagnostics and credential refresh, Windows repository paths, and metadata robustness.

See OCI registry mirroring and Events.

1.6

1.6.1 Trivy-Based SBOM Artifact Generation

zot now supports generating SBOM artifacts during Trivy CVE scans and publishing them as OCI referrers for scanned images.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

For configuration and usage details, see CVE scanning.

1.6.2 Storage: Azure Backend, Fast Restart, and Blob Redirects

This release adds multiple storage improvements:

- Added Azure Blob Storage support as a remote `storageDriver` backend (`name: "azure"`)
- Added optional `storage.fastRestart` to reduce startup time by skipping storage walk when safe
- Added optional `storage.redirectBlobURL` for redirecting blob pulls to backend signed URLs

For details and examples, see Storage.

1.6.3 Authentication and Authorization Enhancements

- Added authorization support for GitHub team memberships in OpenID login flows
- Added metrics endpoint anonymous access controls with stricter authz behavior for authenticated users not in metrics ACL
- Added metrics path validation hardening for safer configuration

See Authentication and authorization and Monitoring.

1.6.4 Event Payload Metadata Improvements

The events extension now includes actor and request metadata in webhook payloads when request context is available.

See Events.

1.6.5 zli Configuration UX

`zli` now supports default named configurations so commands can run without explicitly passing `--config` or `--url` when a default is set.

See zli user guide.

1.6.6 Bug fixes

Many bug fixes around storage behavior, authentication and authorization flows, metrics, CVE reference links, and dependency updates.

1.7

1.7.1 OIDC Logout and Identity Mapping

zot now supports OpenID Connect RP-Initiated Logout and includes improvements to identity handling and claim-mapping logs.

```
{
  "issuer": "https://iam.example.com",
  "scopes": ["openid", "profile", "email", "groups"]
}
```

Authentication mapping also adds support for OpenID groups claims, making it easier to align external identity providers with zot access policies.

```
{
  "claimMapping": {
    "username": "preferred_username",
    "groups": "groups"
  }
}
```


Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.7.2 Conditional Authorization via CEL

Authorization policies can now use conditional expressions through CEL (Common Expression Language), enabling more context-aware access control decisions.

```
{
  "conditions": [
    {
      "expression": "req.time < timestamp(\"2099-12-31T23:59:59Z\")",
      "message": "alice's prod access expires end of 2099"
    }
  ]
}
```

1.7.3 Blob Transfer and Upload Handling Improvements

This release includes multiple API and distribution improvements:

- Added support for multipart range blob pulls and multipart download enhancements
- Improved upload range handling (including proper `416` responses for invalid ranges)
- Recognized Docker Compose/Buildx user agents in the v2 auth challenge workaround

1.7.4 HTTP Read/Write Timeout Configuration

HTTP read and write timeouts are now configurable for both the API server and metrics exporter, with defaults set to `60s`.

Use larger values for environments that handle large images or slower client/network paths. You can also set the timeout values to `0` to disable timeouts (infinite timeout) when required.

For configuration examples and behavior details, see Security Posture: HTTP read/write timeout configuration.

1.7.5 zli and Configuration UX Enhancements

Several usability updates were added to `zli`:

- Improved CVE diff output
- Added a typed `~/.zot` config layer with strict validation
- Added config management commands: `list`, `show`, `get`, `set`, and `reset`

1.7.6 Observability, Signing, and UI Updates

- Added Prometheus metrics for garbage collection
- Added support for cosign bundle metadata
- Updated UI dependencies (`zui`) and related integration pieces

1.7.7 Bug fixes

Many bug fixes around authentication, sync filtering, transfer reliability, lint/CI, dependency updates, and other miscellaneous improvements.

1.8

1.8.1 Repository Quota Enforcement

zot now includes repository quota enforcement middleware in the API layer, helping limit repository growth according to configured quota policies.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

Configuration summary (from PR #3923): set `storage.maxRepos` to cap how many repositories can be created. When the limit is reached, pushes that create a new repository are rejected (HTTP 429), while pushes to existing repositories continue to work. Setting `maxRepos` to `0` (or omitting it) disables enforcement.

```
{
  "storage": {
    "maxRepos": 10
  }
}
```

1.8.2 Multi-tag Push Support

zot now supports pushing multiple tags for a single manifest, improving tag management workflows and reducing duplicate push operations.

`PUT /v2/<name>/manifests/<digest>?tag=1.2.3&tag=1.2&tag=1&tag=latest`

1.8.3 Configuration Schema Export

A new schema command (`zot schema`) can now dump the JSON Schema for zot configuration, making validation and tooling integration easier.

1.8.4 Storage and Authentication Fixes

- Fixed GCS storage handling for root directory prefixing and EOF behavior during walk operations
- Added workaround for Docker client authentication in mixed anonymous-policy setups
- Prevented repository update flows from corrupting `index.json` when disk space is exhausted

1.8.5 Security Hardening

Several hardening fixes were included in this release:

- Limited manifest PUT request bodies to 4 MiB
- Limited API key creation request bodies to 4 KiB
- Suppressed `Allow-Credentials` when CORS origin is wildcard
- Removed `InsecureSkipVerify` usage from metrics client TLS

1.8.6 UI and Search Metadata Enhancements

- Added `LastPullTimestamp` and `PushedBy` fields to search/index `ImageSummary`
- Updated UI dependencies (`zui`) and related integration improvements

1.8.7 Bug fixes

Many bug fixes around security, storage, CI workflows, and other miscellaneous features.

1.9

1.9.1 GCS Storage Support

zot now supports Google Cloud Storage (GCS) as a storage backend, expanding the available storage options alongside existing S3-compatible storage solutions.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.9.2 Dynamic TLS Certificate Reloading

TLS certificates can now be automatically reloaded when changed on disk, eliminating the need to restart zot when certificates are renewed. This is particularly useful in environments with short-lived certificates or automated certificate rotation.

1.9.3 Enhanced OIDC and JWT Authentication

Multiple improvements to authentication capabilities:

- **Per-issuer CA support**: Configure custom certificate authorities for individual OIDC issuers
- **AWS Secrets Manager integration**: Support for AWS Secrets Manager for JWT verification key storage
- **Fine-grained JWT expiration**: Configure `exp` claim at the access entry level for more granular token lifetime control

1.9.4 UI and GraphQL Enhancements

- Added `TaggedTimestamp` to `ImageSummary` returned by GraphQL API
- "Last Tagged" timestamp now displayed in the tag details view in zot's UI

1.9.5 Sync Improvements

- **Legacy Cosign tag filtering**: New `SyncLegacyCosignTags` configuration option to selectively skip syncing legacy cosign/SBOM tags
- **Smart OCI conversion**: Skip OCI conversion for already-synced images, improving sync performance

1.9.6 Security Fixes

- Fixed open redirect vulnerability via `callback_ui`
- Ensured "latest" tag authz checks are not skipped during updates

1.9.7 Bug fixes

Many bug fixes around metadata timestamps, Windows path handling, and other miscellaneous features.

1.10

1.10.1 Workload Identity Federation

zot can now authenticate workloads using OIDC ID tokens, enabling secret-less authentication for automated workflows.

See examples/config-bearer-oidc-workload.json for an example configuration.

1.10.2 Bug fixes

Many bug fixes around storage and other miscellaneous features.

1.11

1.11.1 Bug fixes

Fixes a bug where requests having an Authorization header are rejected if basic auth is disabled.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.12

1.12.1 mTLS Configuration

Previously, there were some access control/authz limitations when enabling UI and mTLS for machine clients. Now mTLS authn configuration supports more flexible identity extraction from peer certs so that access control is uniformly enforced.

1.12.2 Bug fixes

Many bug fixes around security, sync and other miscellaneous features.

1.13

1.13.1 Further FIPS 140-3 Improvements

Previously, password-based authentication supported only *bcrypt* (based on Blowfish) which is not FIPS 140 approved. Starting with this release, both SHA256 and SHA512 based hashes are supported. You can use `mkpasswd` utility (available on most Linux distributions) for this purpose.

1.13.2 Test Retention Policies

A `verify-feature` retention subcommand has been added that allows users to preview and validate retention policy changes without running the actual zot server. The command runs GC and retention tasks in dry-run mode for immediate feedback.

```
zot verify-feature retention -l /var/log/zot-retention-check.log -i 1m -t 10m <config-file>
```

1.13.3 Improved Self-Hosted OAuth2 Support

Some self-hosted applications such as GitHub Enterprise require custom auth url, token url and username claim mapping.

```
  "auth": {
      "openid": {
        "providers": {
          "github": {
            "clientid": <client_id>,
            "clientsecret": <client_secret>,
            "authurl": "https://github.company.com/login/oauth/authorize",     // Custom GHE authorization endpoint
            "tokenurl": "https://github.company.com/login/oauth/access_token", // Custom GHE token endpoint
            "scopes": ["read:org", "user", "repo"],
            "claimMapping": {
              "username": "preferred_username"                                 // Custom claim mapping
            }
          }
        }
      }
    }
```

1.13.4 UI Improvements

zot's UI has been migrated to use vite which helps with startup times and long-term project maintenance.

1.13.5 Bug fixes

Many bug fixes around security, sync and other miscellaneous features.

1.14

1.14.1 zot as a Golang Library

zot was originally intended as an application although implemented in golang. Starting from this release zot can also be used as a golang library.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

```
import zotregistry.dev/zot/v2
```

However, note that this aspect will be best-effort only.

1.14.2 Bug fixes

Some minor bug fixes.

1.15

1.15.1 Garbage Collection and Retention

Untagged manifests are now removed by default in garbage collection unless they are referenced by indexes or artifacts. Previously they were only removed if the retention policies settings were present in the zot configuration. If unspecified, the configuration option for retention delay (used for untagged manifests and orphan referrers), now defaults to the GC delay instead of 24 hours.

1.15.2 FIPS 140-3 Compliance

On Linux, zot can be deployed in FIPS 140-3 mode (in terms of cryptographic protocols used) by setting the environment variable `GODEBUG="fips140=only"`.

Note that GODEBUG=fips140=on and only are not supported on OpenBSD, Wasm, AIX, and 32-bit Windows platforms.

1.15.3 Bug fixes

Update the GraphQL playground version to 5.2, as the old one had broken online dependencies.

1.16

1.16.1 Bug fixes

Some bug fixes in sync mirroring and garbage collection.

1.17

1.17.1 zot OCI Container Images for FreeBSD

FreeBSD community has been releasing OCI images to enable their container ecosystem.

zot is now releasing OCI images to deploy and run as native FreeBSD containers. For example, `ghcr.io/project-zot/zot-freebsd-amd64:latest`.

1.17.2 Bug fixes

Some minor bug fixes.

1.18

1.18.1 Liveness and Readiness Endpoints for Kubernetes Deployments

New HTTP endpoints are added for improved Kubernetes deployments - `/livez`, `/readyz` and `/startupz`. Helm chart has been updated.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.18.2 OpenID Credentials Stored in Files

Previously OpenID credentials were specified inline in zot configuration. Now, they can be loaded from a separate file and the file can be constructed as a Kubernetes `Secret`.

1.18.3 Bug fixes

Some bug fixes around FreeBSD builds.

1.19

1.19.1 Gitlab Social Login

Gitlab login is now supported in zot UI.

1.19.2 Events Extension Token and Custom HTTP Header Support

Earlier, events extension for HTTP sink supported only basic authentication. Now token authentication is supported. Furthermore, you can also include customer HTTP headers to enable sink side routing, etc.

1.20

1.20.1 Bug fixes

Some bug fixes around authentication and authorization.

1.21

1.21.1 Event generation

zot can now generate registry-significant events that can be published to http or nats endpoints.

1.21.2 Improved `docker` Support in UI

Native docker images are now correctly displayed in the Web UI.

1.21.3 Redis driver Support

Previous releases supported a local BoltDB or a remote DynamoDB database in order to store image and blob metadata. This release now includes support for *Redis*.

1.21.4 AWS ECR Sync Support With Temporary Token Authentication

AWS ECR can now be used as the upstream registry to mirror from and the configuration allows for an authentication helper.

1.21.5 Sync Exclude Regex

While mirroring from an upstream registry, it is now possible to exclude images with a regex pattern.

1.21.6 Windows Binaries

Native Windows binaries (both amd64 and arm64) are now distributed with the release.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.21.7 Bug fixes

The following security issue has been fixed in this release.

`GHSA-c37v-3c8w-crq8/CVE-2025-48374`

1.22

1.22.1 Compatibility with other image schema types

A new configuration setting allows you to store images using the schema Docker Manifest v2 Schema v2. With this setting enabled, zot makes no modifications to the image's manifest or digest. Such modifications would otherwise break the image's signature and attestations.

1.22.2 Bug fixes

The following security issue has been fixed in this release.

`GHSA-c9p4-xwr9-rfhxi/CVE-2025-23208`

1.23

1.23.1 Scale-out cluster

You can build a scale-out cluster (proxy/shard based on repository name). Scale-out cluster is compatible with "sync" feature.

1.23.2 Bug fixes

The following security issue has been fixed in this release.

`GHSA-55r9-5mx9-qq7r/CVE-2024-39897`

1.24

This is a maintenance release with minor bug fixes.

1.25

This is a maintenance release with minor bug fixes.

1.26

1.26.1 CVE Query Enhancements

It is now possible to bisect CVEs (`zli cve diff`) between two image tags/versions in the same repository. Furthermore, a CVE query for a particular image tag can return a detailed description of CVEs.

1.26.2 Documentation for "Immutable Image Tags"

A new article has been added to document how image tags can be made immutable.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.26.3 Cross-repo tag search in UI

You can now search for a tag across all repos by starting your query as ':' in the UI, which will return all images that have that tag.

1.26.4 Support for ORAS Artifacts removed

OCI distribution spec 1.1.0 has added support "artifacts" which is likely to gain wider adoption. ORAS artifacts are not widely used or supported.

```
:warning:  Support is removed starting from this version.
```

1.27

1.27.1 Support for hot reloading of LDAP credentials file

Since v2.0.0, LDAP credentials have been specified in a separate file. Starting with this version, the file is watched and changes applied without restarting zot.

1.27.2 Bugfixes and performance improvements

Under some configurations, zot consumes significant CPU and memory resources. This has been fixed in this release.

1.28

1.28.1 Updated OCI support

- Support is added for OCI Distribution Spec v1.1.0-rc3 and OCI Image Spec v1.1.0-rc4. The OCI changes are summarized here. These specifications allow arbitrary artifact types and references so that software supply chain use cases can be supported (SBOMs, signatures, etc). Currently, `oras` and `regclient` support this spec.
- For a demonstration of an end-to-end OCI artifacts workflow, see Software Provenance Workflow Using OCI Artifacts.

  ⚠️ Support is deprecated for earlier OCI release candidates.

1.28.2 Built-in UI support

- Using the new zot GUI, you can browse a zot registry for container images and artifacts. The web interface provides the shell commands for downloading an image using popular third-party tools such as docker, podman, and skopeo.

1.28.3 Support for social logins

- Support is added for OpenID authentication with GitHub, Google, and dex.

1.28.4 Group policies for authorization

- When creating authorization policies, you can assign multiple users to a named group. A group-specific authorization policy can then be defined, specifying allowed access and actions for the group.

  ✏️ **Configuration syntax change**: In the previous release, authorization policies were defined directly under the `accessControl` key in the zot configuration file. With the new ability to create authorization groups, it becomes necessary to add a new `repositories` key below `accessControl`. Beginning with zot v2.0.0, the set of authorization policies are now defined under the `repositories` key.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.28.5 Signature verification

- The validity of an image's signature can be verified by zot. Users can upload public keys or certificates to zot.

1.28.6 LDAP credentials stored separately from configuration

- The LDAP credentials are removed from zot's LDAP configuration and stored in a separate file. See zot's LDAP documentation.

  ⚠️ This LDAP configuration change is incompatible with previous zot releases. When upgrading, you must reconfigure your LDAP credentials if you use LDAP.

1.28.7 Storage deduplication on startup

- Deduplication, a storage space saving feature, now runs or reverts at startup depending on whether the feature is enabled or disabled. You can trigger deduplication by enabling it and then restarting zot.

1.28.8 Retention policies

- To optimize image storage, you can configure tag retention policies to remove images that are no longer needed.

1.28.9 CVE scanning support for image indexes

- The `trivy` backend now supports vulnerability scanning of image indexes. Previously, only images were scanned.

1.28.10 Bookmarks

- In the zot GUI, you can bookmark an image so that it can be easily found later. Bookmarked images appear in search queries when the bookmarked option is enabled.

1.28.11 Ability to delete tags from the UI

1.28.12 Command line search

- The `zli search` command allows smart searching for a repository by its name or for an image by its repo:tag.

1.28.13 Search by digest

- You can perform a global search for a digest (SHA hash) using either the UI or the CLI. This function is useful when an issue is found in a layer that is used by multiple images. In the UI Search box, for example, begin typing `sha256:` followed by a partial or complete digest value to see a dropdown list of images that contain the layer with the digest value.

1.28.14 GraphQL support for search

- A GraphQL backend server within zot's registry search engine provides efficient and enhanced search capabilities. In addition to supporting direct GraphQL queries through the API, zot hosts the GraphQL Playground, which provides an interactive graphical environment for GraphQL queries.

1.28.15 Scheduling of background tasks

- You can adjust the background scheduler based on your deployment requirements for tasks that are handled in the background, such as garbage collection. See Configuring zot.

1.28.16 Performance profiling for troubleshooting

- You can use zot's built-in profiling tools to collect and analyze runtime performance data.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

1.28.17 Binaries for FreeBSD

- zot binary images are available for the FreeBSD operating system. Supported architectures are amd64 and arm64.

  ✏️ zot container images for FreeBSD will be available in a future release.

1.29

1.29.1 Remote-only Storage Support

- The two types of state (images and image metadata) can both now be on remote storage so that zot process lifecycle and its storage can be managed and scaled independently.

1.29.2 Digest Collision Detection During Image Deletion

- When two or more image tags point to the same image digest and the image is deleted by digest causes data loss and dangling references. A new behavior-based policy called *detectManifestCollision* was added to prevent this.

November 2, 2026

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

2. General

2.1 Concepts

2.1.1 What is zot?

👉 **zot** is a production-ready, open-source, vendor-neutral container image registry server based purely on OCI standards.

Two broad trends are changing how we build, distribute, and consume software. The first trend is the increasing adoption of container technologies. The second trend is that software solutions are being composed by combining elements from various sources rather than being built entirely from scratch. The latter trend raises the importance of software provenance and supply chain security. In both trends, zot intends to play an important role by providing a production-ready, open-source, vendor-neutral container image registry server based purely on OCI standards.

What is an OCI image registry?

An OCI image registry is a server-based application that allows you to store, manage, and share container images. A developer uploads (pushes) an image to the registry for distribution. Users can then download (pull) the image to run on their systems. The OCI Distribution Specification, published by the Open Container Initiative (OCI), defines a standard API protocol for these and other image registry operations.

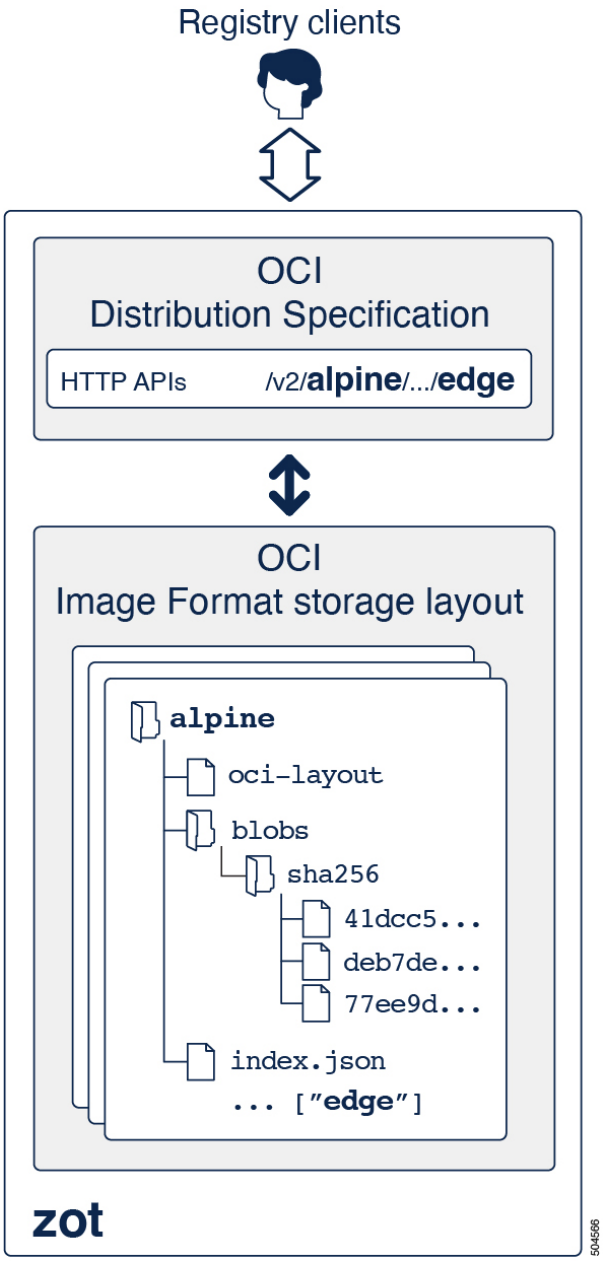
An image registry can be a part of your continuous integration and continuous deployment (CI/CD) pipeline when you host zot on your public or private server. In its minimal form, you can also embed a zot registry in a product. In either case, zot provides a secure software supply chain for container images.

2.1.2 Why zot?

👉 zot = OCI Distribution Specification + OCI Image Format

At its heart, zot is a production-ready, vendor-neutral OCI image registry with images stored in the OCI image format and with the OCI distribution specification on-the-wire. zot is built for developers by developers, offering features such as minimal deployment using a single binary image, built-in authentication and authorization, and inline garbage collection and storage deduplication.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.



Some of the principal advantages of zot are:

- Open source
- OCI standards-only both on-the-wire and on-disk
- Clear separation between core distribution spec and zot-specific extensions
- Software supply chain security, including support for cosign and notation
- Security hardening
- Single binary with many features built-in
- Suitable for deployments in cloud, bare-metal, and embedded devices

zot fully conforms to the OCI Distribution Specification.

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

The following table lists additional advantages of zot:

Distribution Spec conformance	yes
CNCF project	accepted as a Sandbox Project
License	Apache 2.0
On-premises deployment	yes
OCI conformance*	yes
Single binary image*	yes
Minimal build*	yes
Storage Layout	OCI v1 Image Layout
Authentication	built-in
Authorization	built-in
Garbage collection	inline
Storage deduplication	inline
Cloud storage support	yes
Delete by tag	yes
Vulnerability scanning	built-in
Command line interface (cli)	yes
UI	yes
External contributions	beta available
Image signatures	built-in

✏️ * The **minimal build** feature is the ability to build a minimal Distribution Spec compliant registry in order to reduce library dependencies and the possible attack surface.

May 18, 2023

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.

2.2 Summary of Key Features

- Conforms to OCI distribution spec APIs
- Uses OCI image layout for image storage
- Can serve any OCI image layout as a registry
- Single binary for **all** the features
- Doesn't require *root* privileges
- Clear separation between core dist-spec and zot-specific extensions
- Supports container image signatures - cosign and notation
- Supports helm charts
- Behavior controlled via configuration
- Binaries released for multiple operating systems and architectures
- Supports advanced image queries using *search* extension
- Supports image deletion by tag
- Currently suitable for on-premises deployments (e.g. colocated with Kubernetes)
- Compatible with ecosystem tools such as skopeo and cri-o
- Vulnerability scanning of images
- TLS support Authentication via:
- TLS mutual authentication
- HTTP *Basic* (local *htpasswd* and LDAP)
- HTTP *Bearer* token
- Supports Identity-Based Access Control
- Supports live modifications on the configuration file while zot is running (Authorization configuration only)
- Inline storage optimizations:
- Automatic garbage collection of orphaned blobs
- Layer deduplication using hard links when content is identical
- Data scrubbing
- Serve multiple storage paths (and backends) using a single zot server
- Pull and synchronize from other dist-spec conformant registries
- Supports rate limiting including per HTTP method
- Metrics with Prometheus
- Using a node exporter in case of minimal zot
- Swagger based documentation
- zli: command-line client support
- zb: a benchmarking tool for dist-spec conformant registries
- Released under Apache 2.0 License

September 13, 2023

Copyright © 2019 - 2025 The zot project authors, All Rights Reserved.